

UNIVERSITAT POLITÈCNICA DE VALÈNCIA

MIARFID Official Master's Degree in Artificial Intelligence, Pattern Recognition and Digital Imaging

Master Universitario en Inteligencia Artificial, Reconocimiento de Formas e Imagen Digital
Universidad Politécnica de Valencia

FARM: A Formal Approach to Reward Machine Generation in Reinforcement Learning

DEGREE FINAL WORK

Master in Artificial Intelligence, Pattern Recognition and Digital Image

Author: Miquel Gómez

Course 2026-2027

Resum

La construcció automàtica de màquines de recompensa (*Reward Machines*, RM) en l'aprenentatge per reforç s'ha explorat recentment amb grans models fundacionals, que generen RM directament a partir de descripcions de tasques en llenguatge natural. Tot i ser prometedors, aquests mètodes no ofereixen garanties formals i depenen d'avaluacions qualitatives, fetes per persones, de la correcció de les RM. En aquest treball proposem FARM (*A Formal Approach to Reward Machine Generation in Reinforcement Learning*), una cadena de processament basada en la lògica temporal lineal (LTL) per a generar RM: les tasques en llenguatge natural es tradueixen primer a fórmules LTL, que després es compilen en màquines de recompensa amb eines de la teoria d'autòmats. Aquest enfocament produeix RM que, per construcció, estan alineades amb l'especificació temporal desitjada. Introduïm un conjunt de mètriques estructurals i de comportament per a avaluar la qualitat de les RM, i avaluem el rendiment de l'aprenentatge per reforç en termes de taxa d'èxit i d'eficiència mostral en la resolució de tasques. En un banc de proves de tasques amb estructura temporal, el nostre mètode basat en LTL produeix RM més fidels a l'especificació de la tasca i condueix a un aprenentatge més ràpid i fiable que les RM generades per un enfocament de l'estat de l'art basat en models fundacionals. Aquests resultats suggereixen que la síntesi formal de RM guiada per la lògica ofereix una alternativa robusta i interpretable als mètodes de generació de recompenses purament neuronals.

Paraules clau: Aprenentatge per reforç; Màquina de recompensa; Lògica temporal lineal; Especificació de tasques en llenguatge natural

Resumen

La construcción automática de máquinas de recompensa (*Reward Machines*, RM) en el aprendizaje por refuerzo se ha explorado recientemente con grandes modelos fundacionales, que generan RM directamente a partir de descripciones de tareas en lenguaje natural. Aunque prometedores, estos métodos carecen de garantías formales y dependen de evaluaciones cualitativas, realizadas por personas, de la corrección de las RM. En este trabajo proponemos FARM (*A Formal Approach to Reward Machine Generation in Reinforcement Learning*), una cadena de procesamiento basada en la lógica temporal lineal (LTL) para generar RM: las tareas en lenguaje natural se traducen primero a fórmulas LTL, que después se compilan en máquinas de recompensa con herramientas de la teoría de autómatas. Este enfoque produce RM que, por construcción, están alineadas con la especificación temporal deseada. Introducimos un conjunto de métricas estructurales y de comportamiento para evaluar la calidad de las RM, y evaluamos el rendimiento del aprendizaje por refuerzo en términos de tasa de éxito y eficiencia muestral en la resolución de tareas. En un banco de pruebas de tareas con estructura temporal, nuestro método basado en LTL produce RM más fieles a la especificación de la tarea y conduce a un aprendizaje más rápido y fiable que las RM generadas por un enfoque del estado del arte basado en modelos fundacionales. Estos resultados sugieren que la síntesis formal de RM guiada por la lógica ofrece una alternativa robusta e interpretable a los métodos de generación de recompensas puramente neuronales.

Palabras clave: Aprendizaje por refuerzo; Máquina de recompensa; Lógica temporal lineal; Especificación de tareas en lenguaje natural

Abstract

The automated construction of Reward Machines (RMs) in Reinforcement Learning has recently been explored using large foundation models, which generate RMs directly from natural language task descriptions. While promising, these methods lack formal guarantees and rely on qualitative, human-evaluated assessments of RM correctness. In this work, we propose FARM: A Formal Approach to Reward Machine Generation in Reinforcement Learning. A pipeline based on Linear Temporal Logic (LTL) for RM generation: natural language tasks are first translated into LTL formulas, which are then compiled into Reward Machines using automata-theoretic tools. This approach produces RMs that, by construction, are aligned with the desired temporal specification. We introduce a set of structural and behavioral metrics to evaluate RM quality, and assess reinforcement learning performance in terms of success rate and sample efficiency in task resolution. On a benchmark of temporally structured tasks, our LTL-based method produces RMs more faithful to the task specification and leads to faster, more reliable learning compared to RMs generated by a state-of-the-art foundation model approach. These results suggest that logic-guided formal RM synthesis offers a robust and interpretable alternative to purely neural reward generation methods.

Key words: Reinforcement Learning; Reward Machine; Linear Temporal Logic; Natural Language Task Specification

Contents

List of Figures

List of Tables

Acknowledgements

Glossary

CHAPTER 1
Introduction

1.1 Motivation

1.2 Objectives

1.3 Expected impact

1.4 Structure of the paper

1.5 Scope and limits of the study

CHAPTER 2
State of the art

2.1 Related work

2.1.1. Reward Machines

2.1.2. ARM-FM

2.1.3. Linear Temporal Logic

2.1.4. Adaptation to human preferences

2.1.5. Conclusions

2.2 Critique of the state of the art

2.2.1. Limited kinds of tasks

2.2.2. No verification beyond a human expert

2.2.3. No robustness across executions

2.2.4. No formal reward structure

2.2.5. No structure to follow, only LLM inference

2.2.6. Critique of ARM-FM

CHAPTER 3
Proposal

3.1 Human preferences

3.2 No human expert review

3.3 Understanding temporal relations

CHAPTER 4

FARM: A Formal Approach to Reward Machine Generation

4.1 Introduction and motivation

We have to give a bit of what we are using and why. Why formal methods / what we want to acchieve with them and similar stuff. why we already stated things about this on the other sections, like on the sota.

4.1.1. Human preferences

4.2 Global overview of the pipeline

4.2.1. Tools for each step

For this I would not show a vertical one but rather make the 3 block (IBM, FLAT, OURS) on 3 columns, still side by side going verticall and linking to each other but yeah WE also need to change the five rectangles to a stacked version, like one card behind the other so they are not side by side.

3 bulletpoints for each of them

4.2.2. IBM nl2ltl

4.3 Step 1 · The language model picks IBM clauses → LTL

4.3.1. IBM nl2ltl

4.3.2. Templates

4.3.3. The four kinds of templates: goals, rules, recurring and conditional

4.3.4. Adaptation to Reward Machine generation (prompt)

4.3.5. Hard and soft clauses

4.4 Step 2 · FL-AT: MONA turns each LTL formula into a DFA

4.4.1. How we use the tool

4.4.2. The redundant node

4.4.3. Recurrent soft-rule templates

4.4.4. Rule-only machines

4.5 Step 3 · FL-AT multiplies the DFAs into one machine

4.5.1. How we use the tool

4.5.2. What was wrong with its reward

4.5.3. Final states

4.6 Step 4 · Our post-process: hard, soft and rewards

4.6.1. Deleting MONA duplicates

4.6.2. Identifying final states

4.6.3. Sending failed states to a rejecting sink

4.6.4. Reward assignment

4.7 Step 5 · The adversarial critic, retries and thinking level

4.7.1. A review step for the final version

4.7.2. LLMs can hallucinate

4.7.3. Different LLMs for generation and critique

4.7.4. Number of iterations

4.7.5. Fixing mistakes

4.7.6. Removing states that cannot occur in the real environment

CHAPTER 5
Limitations

5.1 The task is assumed to be achievable

5.2 Contradictions

5.3 Unknown facts

5.4 Shortest path

5.5 Human language can be misleading

CHAPTER 6
Evaluation methodology

6.1 Empirical evaluation in a real environment

6.2 Qualitative evaluation of the Reward Machines

6.2.1. From traces to rewards

6.2.2. Shape, number of states and useless states

CHAPTER 7
Results

7.1 Comparison between ARM-FM and FARM

CHAPTER 8
Conclusions

CHAPTER 9

Future work

Bibliography

[1] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Oct. 11, 2018. URL: https://arxiv.org/abs/1810.04805.

[2] Yinhan Liu et al. *ROBERTA: A robustly optimized BERT pretraining approach*. July 26, 2019. URL: https://arxiv.org/abs/1907.11692.

[3] Pengfei Liu et al. *Pre-train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing*. July 28, 2021. URL: https://arxiv.org/abs/2107.13586.

[4] Hosein Mohebbi, Ali Modarressi, and Mohammad Taher Pilehvar. *Exploring the Role of BERT Token Representations to Explain Sentence Probing Results*. Apr. 3, 2021. URL: https://arxiv.org/abs/2104.01477.

[5] Fabian Pedregosa et al. *scikit-learn: machine learning in Python — scikit-learn 1.7.2 documentation*. Jan. 1, 2025. URL: https://scikit-learn.org/stable/.

[6] Fabian Pedregosa, Gaël Varoquaux, and Alexandre Gramfort. *LogisticRegression — scikit-learn documentation*. Jan. 1, 2025. URL: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[7] Tony Chen and Tong He. *Using the scikit-learn Estimator Interface — XGBoost documentation*. Jan. 1, 2025. URL: https://xgboost.readthedocs.io/en/stable/python/sklearn_estimator.html.

[8] Fabian Pedregosa and Gaël Varoquaux. *RandomForestClassifier — scikit-learn documentation*. Jan. 1, 2025. URL: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html.

[9] Fabian Pedregosa and Gaël Varoquaux. *SVC — scikit-learn documentation*. Jan. 1, 2025. URL: https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html.

[10] Fabian Pedregosa and Gaël Varoquaux. *CountVectorizer — scikit-learn documentation*. Jan. 1, 2025. URL: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html.

[11] Fabian Pedregosa and Gaël Varoquaux. *TfidfVectorizer — scikit-learn documentation*. Jan. 1, 2025. URL: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.

[12] Wikipedia contributors. *Sigmoid function*. Oct. 2, 2025. URL: https://en.wikipedia.org/wiki/Sigmoid_function.

[13] *Hugging Face – The AI community building the future.* URL: https://huggingface.co/.

[14] *Gemini API in Vertex AI quickstart*. URL: https://cloud.google.com/vertex-ai/generative-ai/docs/start/quickstart?usertype=adc.

[15] Maxwell Zeff. "Gemini 2.5 Pro is Google's most expensive AI model yet". In: (Apr. 4, 2025). URL: https://techcrunch.com/2025/04/04/gemini-2-5-pro-is-googles-most-expensive-ai-model-yet/.

[16] *tiktoken*. Dec. 15, 2022. URL: https://pypi.org/project/tiktoken/0.1.1/.

[17] *google-bert/bert-base-uncased · Hugging Face*. URL: https://huggingface.co/google-bert/bert-base-uncased.

[18] *FacebookAI/roberta-base · Hugging Face*. URL: https://huggingface.co/FacebookAI/roberta-base.

[19] *microsoft/deberta-v3-base · Hugging Face*. URL: https://huggingface.co/microsoft/deberta-v3-base.

[20] *classla/xlm-roberta-base-multilingual-text-genre-classifier · Hugging Face*. URL: https://huggingface.co/classla/xlm-roberta-base-multilingual-text-genre-classifier.

[21] *FacebookAI/roberta-large-mnli · Hugging Face*. URL: https://huggingface.co/FacebookAI/roberta-large-mnli.

APPENDIX A

Sustainable Development Goals

APPENDIX B

AI usage

APPENDIX C

Development tools

APPENDIX D

IBM template examples

APPENDIX E
Formatting examples

This appendix collects formatting examples for figures, tables, plots and algorithms. It will be removed before submission.

Figure **??** shows a single image.

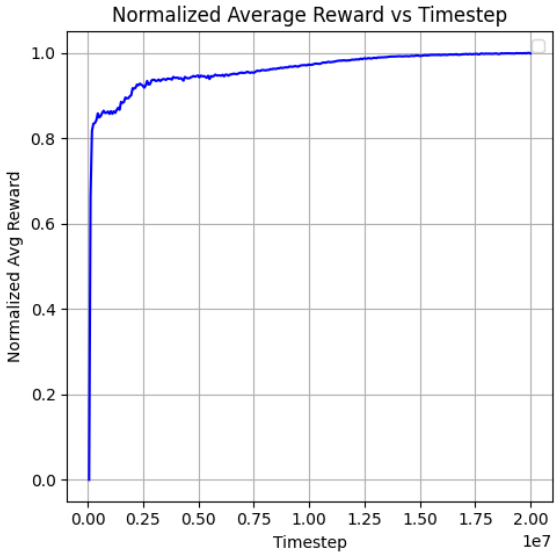
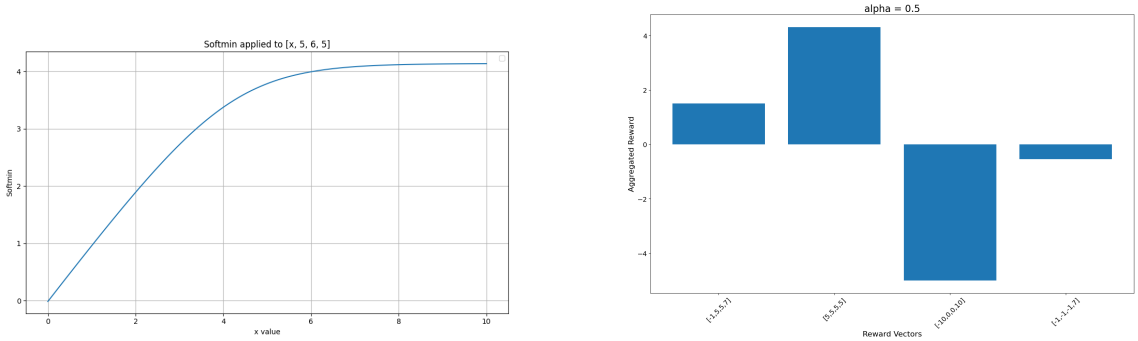


Figure E.1: Example of a single image.

Figure **??** shows two subfigures side by side, Figures **??** and **??**.



(a) First subfigure. **(b)** Second subfigure.

Figure E.2: Example of two subfigures side by side.

Figure **??** shows a diagram drawn with TikZ.

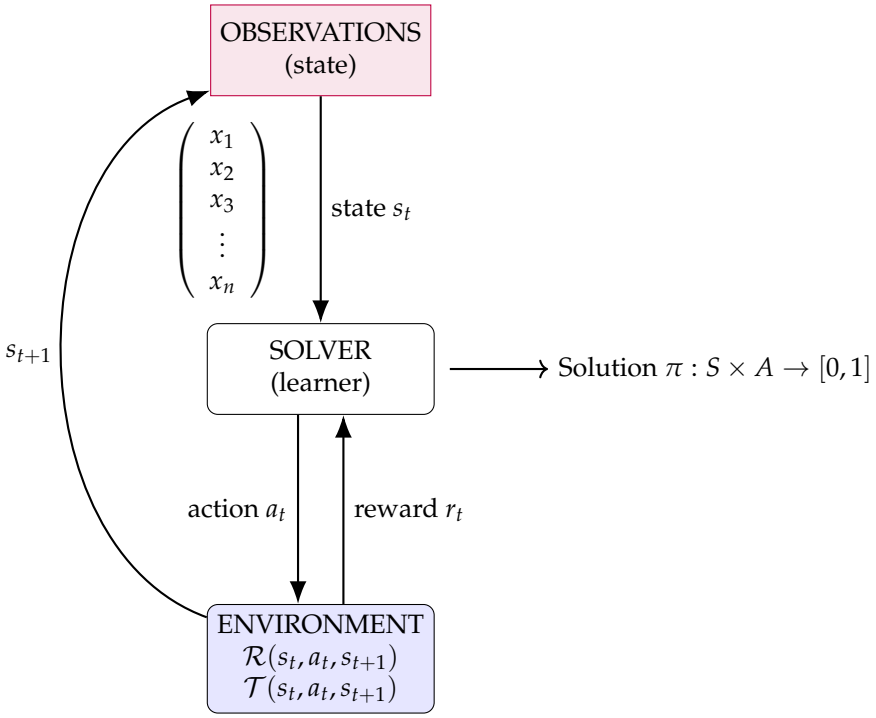
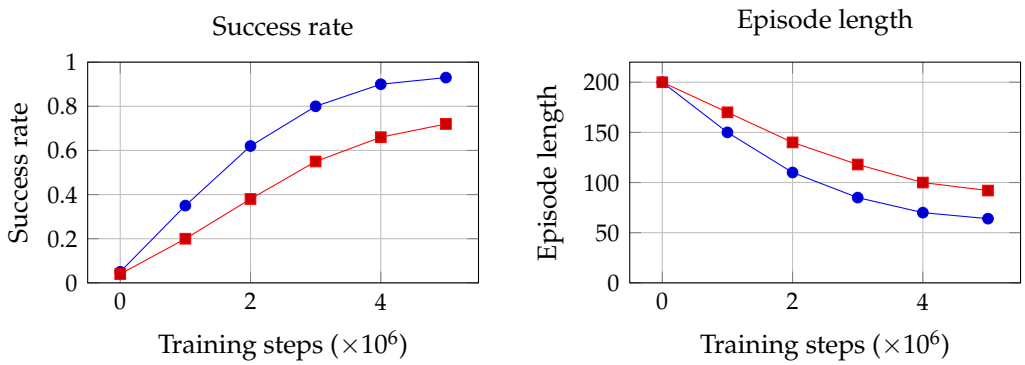


Figure E.3: Example of a TikZ diagram.

Figure **??** shows a multi-panel plot drawn with `pgfplots`.



??

Figure E.4: Example of a two-panel plot with illustrative data: success rate and episode length against training steps for two series.

Table **??** shows a table in `booktabs` format.

Table E.1: Example of a `booktabs` table with illustrative data.

Series	Success rate	Episode length
Series A	0.93	64
Series B	0.72	92

Algorithm **??** shows a block of pseudocode.

Algorithm E.1 Example of a pseudocode block

Input: list of items L, threshold θ
Output: list of kept items L'

```
1:  procedure FILTER(L, θ)
2:      L' ← ∅
3:      for each x ∈ L do
4:          if f(x) > θ then
5:              L' ← L' ∪ {x}                ▷ keep the item
6:          else
7:              continue                     ▷ skip the item
8:          end if
9:      end for
10:     return L'
11: end procedure
```